JAVA THEORY QUESTION BANK

Conceptual questions • Basic, Intermediate, Advanced

Topics included

Access modifiers, primitive data types, variables, operators, Strings, arrays, control flow, classes and objects, constructors and overloading, shallow/deep copying, input and output.

How to use this workbook

Answer in your own words. For code-tracing or design questions, include a short example. Work from Basic to Advanced and revisit questions you cannot yet explain.

BASIC — MODIFIERS AND DATA TYPES

1. What is an access modifier in Java? Explain **public** and **private** with examples.

2. Classify these as keywords or user-defined identifiers: private, age, public, Student, totalMarks.

3. List Java's eight primitive data types and their usual sizes in bits.

4. Compare int, long, and short. Why does byte b = 130; fail to compile?

5. Compare float and double. What do char and boolean represent?

BASIC — VARIABLES AND OPERATORS

6. Differentiate variable declaration, initialization, and reassignment with examples.

7. Explain local, instance, and static variables. Which get default values?

8. Explain the difference between = and ==.

9. Compare prefix increment (++i) and postfix increment (i++).

10. What is the difference between && and &? Explain operator precedence and integer division.

BASIC — STRINGS AND ARRAYS

11. Are Java Strings mutable or immutable? Explain.

12. Differentiate == and equals() when comparing Strings.

13. What is an array? Why is its length fixed after creation?

14. How do you get the length of a String versus an array?

15. What happens when an invalid array index is accessed?

INTERMEDIATE — CONTROL FLOW

16. Compare if/else with switch. When might each be suitable?

17. Compare while and do-while. Which executes at least once?

18. Describe the initialization, condition, and update parts of a traditional for loop.

19. Java has an enhanced for loop. Explain how for (int x : values) is similar to Python's for x in values.

20. What is switch fall-through? Explain break and continue.

INTERMEDIATE — CLASSES, OBJECTS, CONSTRUCTORS

21. Define a class and an object, using a real-world example.

22. What are fields and methods?

23. What is a constructor, and how does it differ from an ordinary method?

24. When does Java provide a default constructor? Is it supplied if you declare your own constructor?

25. Explain constructor overloading. What must differ between overloaded constructors?

26. What does this mean in a constructor? Can constructors have a return type?

27. Explain encapsulation using private fields and public methods.

INTERMEDIATE — SHALLOW AND DEEP COPYING

28. What is a reference variable? Does assigning one object variable to another copy the object?

29. Explain shallow copying using an object that contains an array field.

30. Explain deep copying and how it differs from shallow copying.

31. Why can changing a nested object in a shallow copy affect the original object?

32. Why is a copy constructor a pattern programmers implement rather than a special constructor automatically generated by Java?

INTERMEDIATE — INPUT AND OUTPUT

33. What is the difference between print(), println(), and printf()?

34. What is Scanner? Name methods for reading an integer, a decimal, and a full line.

35. Why can nextLine() seem to skip input after nextInt()?

36. Why might System.console() return null when running a program in an IDE?

37. How would you format a decimal to two places with printf()?

ADVANCED — COMBINED REASONING

38. A class has a primitive field, a String field, and an array field. Explain which parts are shared in a simple shallow copy.

39. Design a constructor that validates input. What should happen when invalid data is supplied?

40. Explain a safe way to compare Strings and why == is unreliable for text equality.

41. Describe a testing strategy for an array-processing program that reads input and prints a result.

42. Explain how a loop boundary error can cause an array index exception or skip an element.

Completion checklist

■ I can explain the concept. ■ I can compare related concepts. ■ I can trace code. ■ I can explain common errors.